

东南大学 · 网络空间安全学院

DeFuzz：不变量引导的编译器防御机制测试

Invariant-Guided Agentic Testing
of Compiler-Instrumented Defenses

张景耀 · 指导教师：徐坚皓

小组分享 · 2026.09

研究背景：编译器插进去的保护，要单独验证有没有真生效

背景



研究对象与判定难点

研究对象

编译器自动插入的保护：栈金丝雀、CFI、PAC 等

实现位置

一半在编译器通用部分，一半在各架构的代码生成

核心判断

插了保护代码 ≠ 真的受保护

难在哪

通用逻辑共享，但每个架构生成的结果要分别验

我们查的是编译出来的程序有没有兑现保护承诺，不是查应用代码里还有没有漏洞。

来源：论文 §I、§II-A

问题动机：程序跑得对，保护其实已经失效

背景



CVE-2023-4039 • AArch64 栈保护

本该成立的规则

从缓冲区一路往上覆盖，必须先碰到金丝雀，才可能碰到返回地址

实际的栈布局

返回地址被放在缓冲区上面、金丝雀下面

于是悄悄失效

返回地址先被改掉，函数结束时金丝雀检查照常通过

程序不崩、输出也没差别，所以过去测编译器的两种办法都判不出来。

来源：论文 §I-A、§II-D、§II-F（CVE-2023-4039；判不出来的缺口）

三项挑战：不变量生成、基于不变量做测试、ISA 差异

挑战

C1 不变量生成

安全规则散在文档、源码注释、ABI 规范和历史补丁里，没有一条现成能让程序去判；而编译器现在的行为本身可能就是错的，不能当标准。

DeFuzz 的对应设计

分段读语料，逐条写成规则，每条都要指明出处

C2 基于不变量做测试

有规则也不一定测得出来：既要有能看出违规的检查器，也要有能走到那条保护路径的测试程序——4039 只在“动态大小数组 + 正常返回”时才暴露。

DeFuzz 的对应设计

锁定一条规则，照着覆盖率和判定结果反复改写测试程序

C3 ISA 差异

同一条规则，在不同架构上要看的东西不一样：栈怎么摆、寄存器怎么用、指令什么语义都不同，一条规则得为每个架构分别写检查器。

DeFuzz 的对应设计

一条规则 × 一个架构写一个检查器，用哪个由查表决定

三件事是连着的：规则写不出来就没目标，检查器和程序配不上就判不准，架构对不齐就不可信。

来源：论文 §I-B（三项耦合挑战）、2026-09-11 关于“按架构分别写检查器”的修订

研究目标与研究范围

挑战

研究目标（论文的四项贡献）

- 把散落的安全要求写成一条条带出处的规则
- 围绕规则做定向测试：过程可追溯，部件能逐个关掉做对比
- 一条规则按架构分别落成检查器，只给确定的对错
- 在 GCC / LLVM 的多个架构上真找出问题，并被上游确认

研究范围（哪些算、哪些不算）

- 前提：漏洞本身可触发、保护已打开、攻击不超出机制自己的承诺
- 算：编译过了、功能测试也过，但编出来的产物违反了规则
- 不算：应用层漏洞、源码里主动绕过、硬件与侧信道、优化与防御的交互
- 版本：语料固定 GCC 16.1；结论跟编译器版本和目标配置绑定

要的是能判、能复现、能换架构复用；不做端到端利用，也不承诺数学证明。

来源：论文 §I-D（Contributions）、§II-C（Threat Model and Scope）

技术路线总览：测什么、怎么判、怎么找

技术路线



测什么 · 解 C1

先攒出一份规则清单，每条写明出处和什么时候适用

怎么判 · 解 C3

一条规则按架构各写一个检查器，只给确定的结论

怎么找 · 解 C2

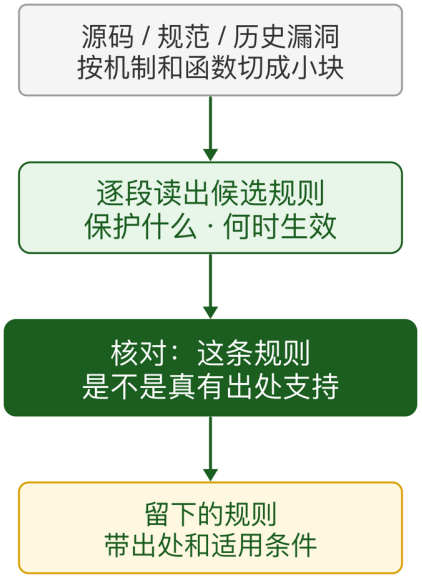
盯住一条规则，让 Agent 照着反馈反复改写测试程序

三件事扣成一个环：规则说清测什么，检查器说清怎么判，固定流程说清怎么找。

来源：论文 §III（System Overview）、§VIII（Implementation）

测什么：把散落的安全要求写成一条条规则

技术路线



一条规则长什么样（用 4039 那条举例）

防御机制	AArch64 栈保护
什么时候适用	保护已打开；函数里有动态大小的分配
规则本身	从缓冲区覆盖到返回地址，必先经过金丝雀
怎么看出来	返回地址被改了，金丝雀还是原来的值
依据出处	CVE-2023-4039 的补丁、调用约定里的栈布局要求

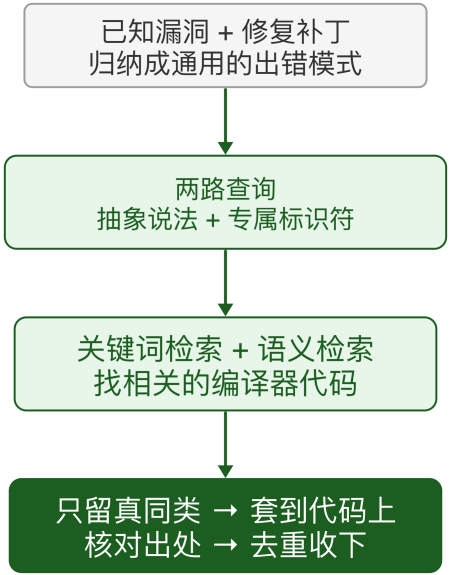
“怎么看出来”只写现象，不写用什么手段去查；这样换架构时只改查法，规则本身不动。

语料按机制和函数切成小块逐段读，每条候选都要能指回出处，还要查重。

来源：论文 §IV-A / B / C、§V-D

测什么：拿一个已知漏洞当线索，找出同一类毛病

技术路线



四步走（线索：CVE-2023-4039）

① 抽掉细节	把这个漏洞归纳成一句通用的出错模式
② 找同类	只留真的同一种错，名字像但机理不同的直接否掉
③ 套到代码上	在具体的目标代码里写成一条新规则
④ 核对去重	看引用的依据是否真能支持这条规则，再去掉重复

实测：24 条线索、4,496 个代码块，关键词检索命中 9 条、语义检索命中 7 条，交集只有 5——两种检索不能互相替代。

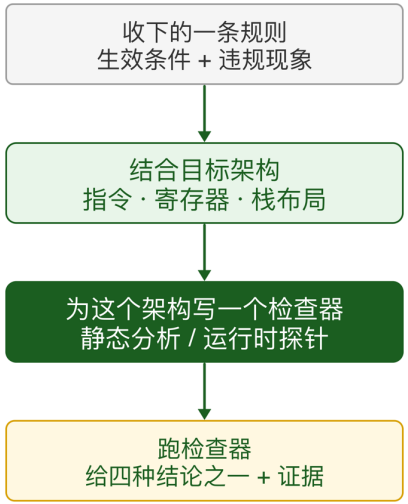
刻意不按函数名、符号名去搜，否则只能在原机制里打转；确认的新问题会回流成新线索。

来源：论文 §V-A / B / C / D、Table I

怎么判：检查器只给四种结论

技术路线

检查器的输入：编译产物 + 目标架构 + 一条规则；输出只有四种结论



四种结论各是什么意思

Pass 通过	条件成立，这一轮没发现违规
Fail 违规	确定违规，带着证据转去做精简
NA 不适用	保护没开、没触发，或这一格判不了；统计时忽略
Error 出错	编译或分析本身失败，只作排查，不算漏洞

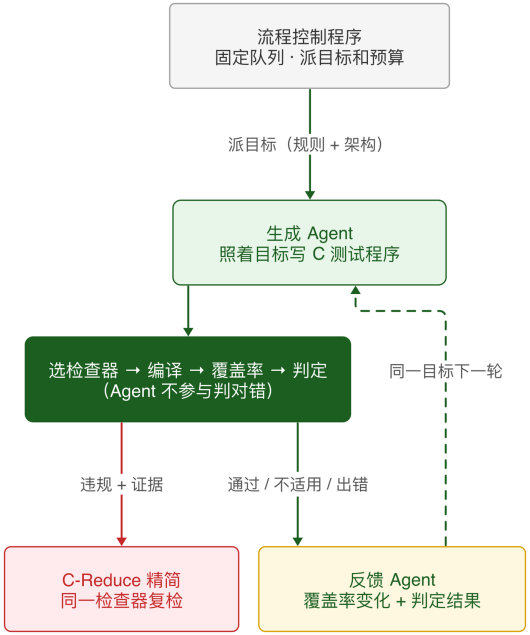
一条规则在各架构的检查器里，任一个报违规就算这个机制被违反；同一架构上 GCC 和 LLVM 可以共用检查器，跨架构不共用。

用哪个检查器是查表定的，不是让模型去挑；“判不了”绝不能算成“保护成功”。

来源：论文 §VI-A / B / C（判定函数、四种结论、机制级汇总）

怎么找：流程由程序控制，Agent 只负责写测试程序

技术路线



分工与纪律

流程提前定好，写在程序里

开跑前就把目标排成固定队列，Agent 不挑规则也不挑架构。

生成 Agent

只能查源码和规则，只负责写 C 测试程序。

反馈 Agent

单独开一轮、不带历史，把覆盖率变化和判定结果变成改写建议。

精简

主力是 C-Reduce；每个候选都要重新编译、由同一个检查器复检。

判对错这一步完全没有模型参与：所有“违规”都来自程序化的检查器。

来源：论文 §VII-A / B / C / D、Algorithm 1

串起来看：一个测试程序怎么走到一次确定的判定

技术路线

测试程序必须保留的结构（示意，不是可利用的 PoC）

```
extern void use(char *, unsigned);

void f(unsigned n) {
    char buf[n];
    use(buf, n);
}
```

buf < 返回地址 < guard → Fail

1. 先确认真走到保护路径

保留动态大小数组和正常返回，别被优化掉。

2. 按架构选检查器

查表选中对应实现，解析出缓冲区、返回地址、金丝雀的位置。

3. 判定

缓冲区 < 返回地址 < 金丝雀 → 判违规。

4. 出证据、做精简

给出相关指令和地址关系；精简后由同一个检查器复检。

解析不出来就报“出错”或“不适用”，绝不猜违规；查的是三者的前后关系，不是某个符号在不在。

来源：论文 §II-D、§VI、§VII-D；检测过程示意

已有成果：三例已上报的静默失效（沿用开题）

结果

保护序列已结束，Canary 却仍在寄存器中

> I consider this a severe security vulnerability for LoongArch, so we
> should backport it.
>
> What do you think?

I agree. I'll push it to trunk and backport if no complains in a few days.

_FORTIFY_SOURCE 长度上限被放宽

tree-ssa-strlen 优化 · PR-125079

把 __strcat_chk 优化成 __stpcpy_chk 时没同步减小长度参数，运行时的安全上限被悄悄放宽。

✓ 已合入 master

x86 IBT 的 endbr 漏检

x86 后端 endbr 判断 · PR-125084

扫描时碰到立即数高位不为零就提前退出，藏在高位的 endbr 被漏掉，代码里混进了可被跳进去的假入口。

⏳ 等待维护者回复

多架构金丝雀残留在寄存器里

通用后备实现 · PR-125045 / 125049

部分架构的通用后备实现返回前没清掉金丝雀寄存器，这个全进程通用的值一直留在寄存器中。

✓ LoongArch 子例已合入

目前范围是 GCC 的多个架构和版本；真实案例能证明动机，但算不出漏检率。

来源：开题答辩 P15 / P16 / P17：现有成果与上游反馈，内容直接沿用

评估设计：四个研究问题和对应指标

结果

研究问题	看什么指标	怎么做
RQ1 真实缺陷	确认的失效数、上游确认数	在 GCC / LLVM 的多个架构上按规则跑
RQ2 规则质量	专家分级结果与打分一致性	抽样评审候选规则和它引用的依据
RQ3 各部件多重要	发现率、耗时、模型调用成本	分别关掉固定流程、覆盖反馈、出处核对
RQ4 换架构灵不灵	各架构分别的发现率与复用边界	同一条规则在多个架构上分开统计

没触发、跑挂了、误报都单独记账，不混进“保护成功”。

四个问题现在都还只是设计；数值等实验，今天不报结果。

来源：论文 §IX（RQ1–RQ4）、§X（Discussion and Limitations）

漏检评估：用已经做好的 100 项人工植入做盲测

结果

DeFuzz_injection_delivery_20260916

100 植入的缺陷　　**50 + 50** GCC / LLVM　　**10** 批次补丁

植入示例 G001 · GCC 栈保护的判定分支

```
else if (defuzz_mutant_enabled("G001")
         && TYPE_MAIN_VARIANT(t)
            == integer_type_node)
    defuzz_mutant_hit("G001");
else
    ret = SPCT_HAS_ARRAY;
```

这个补丁干了什么

打开 G001 后，int 数组不再算进需要保护的情况。

怎么盲测

每次只打开一项；系统看不到补丁、编号和答案。

什么算漏检

在固定预算内没把这项已确认有效的植入找回来。

漏检率 = 没找回的植入数 ÷ 全部有效植入数；植入已经做完，实验还没开始。

来源：植入交付 patches/gcc-batch01.patch 与批次 manifest；代码节选

工作进度：六阶段推进（沿用开题计划）

结果

S1 2026.05 — 07

扩展防御机制覆盖

从金丝雀、长度检查、间接跳转保护，扩展到 BTI / PAC、影子栈、CFI，把静态和运行时检查器补齐。

S2 2026.07 — 09

迁移到 LLVM

把编译、覆盖率适配和开关选项这套东西扩到 LLVM，让跨架构、跨编译器的实验框架对齐。

S3 2026.09 — 12

全机制端到端实验

在 GCC 和 LLVM 的多个版本、多个架构上跑完整实验，同时继续推进向上游上报的闭环。

S4 2026.12 — 2027.02

基线对照评测

和 Csmith、YARPGen、GrayC、HLPFuzz 等已有工具对照，量化覆盖率推进和缺陷发现上的优势。

S5 2027.02 — 04

论文初稿与投稿

整理实验数据和图表，完成学位论文初稿，准备会议 / 期刊投稿。

S6 2027.04 — 06

修订与答辩准备

论文修订、处理外审意见、准备答辩材料；2027.06 学位论文答辩。

当前在 S3：方法定稿、检索实测和 100 项植入都已就位；端到端实验与基线对照还没开展。

一套方案三件事：测什么靠带出处的规则，怎么判靠按架构写的检查器，怎么找靠固定流程的定向生成。

来源：开题答辩 P19：工作进度 S1–S6，阶段划分直接沿用；当前位置按本次进度更新